

JAVA INTERVIEW PREPARATION

CHAPTER 66

STACKS, QUEUES, DEQUES,
AND MONOTONIC STRUCTURES

Senior / Lead Java Developer Textbook Edition

Full reading material - not summarized

Stacks, Queues, Deques, and Monotonic Structures

Senior / Lead Java Developer Reading Chapter

Stacks and queues are often introduced as tiny abstract data types: a stack is last-in, first-out, while a queue is first-in, first-out. That definition is correct but incomplete for a senior interview. The more useful question is what ordering guarantee the structure gives, which operations preserve that guarantee, and what failure or capacity behavior the surrounding system requires.

A stack makes the most recently opened piece of work the next one to finish. A queue preserves arrival order. A deque supports insertion and removal at both ends and can implement either discipline. Monotonic stacks and deques add another invariant: values or priorities remain ordered inside the structure, allowing candidates that can never become useful to be discarded early.

These structures appear far beyond coding exercises. Parser state, request buffering, work scheduling, browser history, breadth-first traversal, rate smoothing, asynchronous handoff, and sliding-window monitoring all depend on the same small set of ordering ideas.

The ordering contract is the real abstraction

Consider three tasks arriving in the order A, B, C.

Stack:	Queue:
push A, push B, push C	offer A, offer B, offer C
top	front rear
C <- removed first	A B C
B	^ removed first
A	

The difference is not an implementation detail. It changes the meaning of the program. A depth-first search uses a stack because it continues with the newest unexplored branch. A breadth-first search uses a queue because it completes the current frontier before moving to the next one. Undo uses a stack because the latest action must be reversed first. A work queue usually preserves arrival order, although production schedulers may deliberately introduce priority, fairness, or deadlines.

A senior answer should therefore start with the semantic requirement. Only then should it choose an implementation.

Java collection choices

For ordinary single-threaded stack, queue, and deque work, [ArrayDeque](#) is usually the first choice. It supports constant-time insertion and removal at both ends in the normal case, avoids one node allocation per element, and can act as either a stack or a queue.

```
Deque<String> stack = new ArrayDeque<>();
stack.push("A");
stack.push("B");
String mostRecent = stack.pop();           // B

Deque<String> queue = new ArrayDeque<>();
queue.offerLast("A");
queue.offerLast("B");
String oldest = queue.pollFirst();        // A
```

Using explicit end names such as `offerLast` and `pollFirst` makes queue direction visible. `push` and `pop` are convenient when the variable is clearly a stack. `ArrayDeque` does not permit `null`; that is useful because `poll` returns `null` to signal an empty deque. It is not thread-safe, so sharing it between threads requires external synchronization or a concurrent alternative.

The legacy `Stack` class extends `Vector`. Its inheritance exposes list operations that weaken the stack abstraction, and its synchronization model rarely matches a modern design. `LinkedList` implements `Deque`, but its per-node allocation, pointer chasing, and weaker cache locality usually make `ArrayDeque` preferable unless linked-node behavior is specifically needed.

The `Queue` API contains paired operations:

| Intent | Exceptional form | Special-value form | |---|---|---| | insert | `add(e)` | `offer(e)` | | remove head | `remove()` | `poll()` | | inspect head | `element()` | `peek()` |

The special-value forms are usually easier at capacity and empty boundaries. A bounded queue can reject `offer` with `false`; a blocking queue may instead wait through `put`. Those are different service behaviors, not interchangeable method styles.

Stacks encode unfinished work

A stack is useful whenever processing creates a nested obligation that must be resolved in reverse order. Balanced delimiters are the simplest example.

```
static boolean balanced(String text) {
    Deque<Character> expected = new ArrayDeque<>();

    for (char c : text.toCharArray()) {
        switch (c) {
            case '(' -> expected.push('(');
            case '[' -> expected.push('[');
            case '{' -> expected.push('{');
            case ')', ']', '}' -> {
                if (expected.isEmpty() || expected.pop() != c) {
                    return false;
                }
            }
            default -> { }
        }
    }
    return expected.isEmpty();
}
```

Storing the expected closing delimiter makes the invariant direct: the top of the stack is exactly what may legally close next. It is clearer than pushing openers and repeatedly translating them later.

Expression evaluation uses the same idea with two kinds of unfinished work: operands and operators. When operators have precedence or parentheses, the implementation delays an operator until it is safe to evaluate it. Recursive descent parsing uses the language call stack for nested grammar rules; an explicit stack is useful when depth may be large or when parsing must pause and resume.

An explicit stack can also replace recursion:

```

static void depthFirst(Node root) {
    if (root == null) return;

    Deque<Node> pending = new ArrayDeque<>();
    pending.push(root);

    while (!pending.isEmpty()) {
        Node node = pending.pop();
        visit(node);

        // Push right first so left is processed first.
        if (node.right != null) pending.push(node.right);
        if (node.left != null) pending.push(node.left);
    }
}

```

The important detail is reverse insertion. Because the stack reverses removal order, children must be pushed in the opposite order from the desired visit order.

Queues preserve a frontier

A queue represents work discovered but not yet processed. In level-order tree traversal, it holds the next frontier.

```

static List<List<Integer>> levels(TreeNode root) {
    if (root == null) return List.of();

    List<List<Integer>> result = new ArrayList<>();
    Deque<TreeNode> queue = new ArrayDeque<>();
    queue.offerLast(root);

    while (!queue.isEmpty()) {
        int levelSize = queue.size();
        List<Integer> level = new ArrayList<>(levelSize);

        for (int i = 0; i < levelSize; i++) {
            TreeNode node = queue.pollFirst();
            level.add(node.value);
            if (node.left != null) queue.offerLast(node.left);
            if (node.right != null) queue.offerLast(node.right);
        }
        result.add(level);
    }
    return result;
}

```

Capturing `levelSize` before the inner loop creates a boundary between the current level and children added for the next one. Reading `queue.size()` as the loop condition would mix those frontiers.

In production, a queue is also a temporal boundary. A producer can hand off work without waiting for full processing, and a consumer can absorb bursts. But a queue does not create capacity. If arrival rate remains above service rate, backlog grows. Little's Law gives the useful relationship $L = \lambda * W$: average items in the system equal arrival rate multiplied by average time in the system. A queue that appears to improve responsiveness may merely hide overload while latency and memory consumption accumulate.

Circular buffers and ring invariants

An array-backed queue uses a circular layout so removed space can be reused without shifting elements.

```
capacity = 8

index:  0   1   2   3   4   5   6   7
data:  [D] [E] [ ] [ ] [ ] [A] [B] [C]
           tail       head

logical order: A, B, C, D, E
```

head identifies the first logical element and **tail** identifies the next insertion slot. Both wrap with modulo arithmetic. Tracking **size** removes the ambiguity between an empty buffer and a full buffer when **head == tail**.

```
final class IntRingBuffer {
    private final int[] elements;
    private int head;
    private int tail;
    private int size;

    IntRingBuffer(int capacity) {
        if (capacity <= 0) throw new IllegalArgumentException("capacity");
        elements = new int[capacity];
    }

    boolean offer(int value) {
        if (size == elements.length) return false;
        elements[tail] = value;
        tail = (tail + 1) % elements.length;
        size++;
        return true;
    }

    OptionalInt poll() {
        if (size == 0) return OptionalInt.empty();
        int value = elements[head];
        head = (head + 1) % elements.length;
        size--;
        return OptionalInt.of(value);
    }
}
```

The core invariant is $0 \leq \text{size} \leq \text{capacity}$. Every successful insertion advances **tail** and increments **size**; every successful removal advances **head** and decrements **size**. A generic implementation should clear removed array slots so obsolete references do not retain objects.

Power-of-two capacities can replace modulo with a bit mask, but that optimization is justified only after measurement and only when the capacity invariant is enforced. Correct wraparound is more valuable than a speculative arithmetic saving.

Bounded queues, overload, and backpressure

An unbounded queue converts sustained overload into memory growth and extreme latency. A bounded queue forces an explicit policy when capacity is exhausted:

- reject immediately;
- block the producer;
- time out;
- discard newest or oldest work;
- coalesce equivalent work;
- redirect to durable storage.

The correct policy depends on the domain. Rejecting an interactive request with a clear overload response can be safer than accepting it and timing out much later. Dropping an old telemetry sample may be acceptable; dropping a payment command is not. Blocking can propagate pressure upstream, but blocking request threads indefinitely can itself exhaust a server.

`BlockingQueue` makes waiting semantics explicit. `ArrayBlockingQueue` has fixed capacity and predictable allocation. `LinkedBlockingQueue` may be bounded but defaults to a very large capacity if none is provided. `SynchronousQueue` stores no element at all: each handoff pairs a producer with a consumer. `PriorityBlockingQueue` is unbounded and priority-ordered rather than FIFO.

Queue size should be treated as an operational control. Capacity, rejection count, queue wait time, processing time, and oldest-item age are more informative together than raw queue length alone.

Capacity calculations should be tied to a latency budget. Suppose 20 workers each complete about 10 jobs per second and the service allows at most 500 milliseconds of queueing before work becomes stale. A rough steady-state queue allowance is about $200 \text{ jobs/second} * 0.5 \text{ seconds} = 100 \text{ jobs}$, with additional room only for a measured and acceptable burst. Choosing 100,000 because memory permits it would allow roughly 500 seconds of wait at the same drain rate. The arithmetic is approximate, but it exposes whether capacity represents deliberate burst absorption or merely delayed failure.

Capacity also interacts with retry traffic. If failures cause every job to be re-enqueued immediately, the effective arrival rate can multiply precisely when service capacity is lowest. Separate delayed retry scheduling from the fresh-work queue, apply jittered backoff, and cap attempts. Otherwise the queue's nominal FIFO contract can turn into retry-driven starvation.

Concurrency changes the choice

Collection interfaces describe ordering; concurrent implementations additionally define atomicity and visibility. Wrapping a sequence of operations around a thread-safe queue does not automatically make the sequence atomic.

```
if (!queue.isEmpty()) {
    Job job = queue.poll();
    // Another consumer may have removed the last job first.
}
```

The safe operation is simply `poll` and then checking its result. This is the same check-then-act race seen throughout concurrent programming.

For non-blocking multi-producer access, `ConcurrentLinkedQueue` provides a scalable unbounded queue. For producer-consumer coordination, blocking queues often express the design more clearly. A `Deque` also supports work-stealing: an owner takes recent tasks from one end while another worker steals older tasks from the other, reducing contention and improving locality.

No queue removes the need to define shutdown. Consumers require a cancellation signal, interruption policy, poison message, or closed-channel abstraction. Systems must decide whether shutdown drains accepted work, persists it, or abandons it. That contract is part of correctness.

Monotonic stacks discard dominated candidates

A monotonic stack stores candidates in increasing or decreasing order. When a new value makes earlier candidates permanently irrelevant, those candidates are removed immediately. Each element is pushed once and popped at most once, which is why many apparently nested algorithms remain $O(n)$.

For the next greater element, keep indices whose answer has not yet been found:

```

static int[] nextGreater(int[] values) {
    int[] answer = new int[values.length];
    Arrays.fill(answer, -1);
    Deque<Integer> unresolved = new ArrayDeque<>();

    for (int i = 0; i < values.length; i++) {
        while (!unresolved.isEmpty()
            && values[unresolved.peek()] < values[i]) {
            answer[unresolved.pop()] = values[i];
        }
        unresolved.push(i);
    }
    return answer;
}

```

The stack contains indices with decreasing values from bottom to top. Indices are stored because the result belongs to a position, and duplicate values must remain distinguishable. Decide deliberately whether equality should pop: < answers “strictly greater,” while <= implements a different contract.

The largest-rectangle-in-a-histogram problem uses a monotonic increasing stack. When a shorter bar arrives, taller bars can no longer extend right, so their maximal areas are finalized. The difficult part is not memorizing code; it is stating what each index represents and exactly which boundaries remain possible.

Monotonic deques maintain a window optimum

A sliding-window maximum can be computed in linear time with a deque of candidate indices.

```

static int[] windowMaximum(int[] values, int width) {
    if (width <= 0 || width > values.length) {
        throw new IllegalArgumentException("width");
    }

    int[] result = new int[values.length - width + 1];
    Deque<Integer> candidates = new ArrayDeque<>();

    for (int right = 0; right < values.length; right++) {
        int expired = right - width;
        if (!candidates.isEmpty() && candidates.peekFirst() <= expired) {
            candidates.pollFirst();
        }

        while (!candidates.isEmpty()
            && values[candidates.peekLast()] <= values[right]) {
            candidates.pollLast();
        }
        candidates.offerLast(right);

        if (right >= width - 1) {
            result[right - width + 1] = values[candidates.peekFirst()];
        }
    }
    return result;
}

```

The deque has two independent rules. Indices outside the window expire from the front. Values dominated by the new element disappear from the back. The front is therefore always the index of the current maximum. Keeping indices, rather than values, makes both expiry and duplicates correct.

```

window values: [4, 2, 2, 5]

candidate deque after 4: [4]
after first 2:          [4, 2]
after second 2:         [4, 2]   older equal 2 discarded
after 5:                [5]      every earlier value dominated

```

This pattern is useful for rolling latency maxima, capacity envelopes, and time-series processing when the window is count-based. Time-based windows additionally need timestamps and rules for late or out-of-order events.

Augmented stacks and composed structures

A minimum stack returns the current minimum in $O(1)$ by storing extra information.

```
record Entry(int value, int minimumSoFar) {}

final class MinStack {
    private final Deque<Entry> entries = new ArrayDeque<>();

    void push(int value) {
        int minimum = entries.isEmpty()
            ? value
            : Math.min(value, entries.peek().minimumSoFar());
        entries.push(new Entry(value, minimum));
    }

    int pop() { return entries.pop().value(); }
    int min() { return entries.element().minimumSoFar(); }
}
```

Duplicating the minimum per entry simplifies duplicate handling and makes each operation local. An alternative two-stack design stores a value on the minimum stack whenever it is less than or equal to the current minimum. Forgetting equality is a classic bug: popping one occurrence would lose a minimum that still exists.

A queue can be built from two stacks. New elements enter an input stack; removals come from an output stack. When the output stack is empty, all input elements move across, reversing their order. Although one operation may move many elements, each element moves at most once between stacks, giving amortized $O(1)$ operations. The amortized qualifier must be stated: an individual removal can still be $O(n)$, which matters for strict tail-latency requirements.

Priority is not FIFO

`PriorityQueue` exposes the least element under its ordering, not the oldest element. Iterating it does not produce sorted order. Equal-priority elements have no general FIFO guarantee unless the application adds a sequence number to the comparator.

This distinction matters in schedulers. Pure priority can starve low-priority work; aging or weighted fairness may be required. Heap internals and heap-selection algorithms belong to the next chapter, but the design lesson belongs here: changing a queue's ordering changes its fairness and service contract.

Production reasoning and failure modes

When a queue-backed service slows down, inspect rates and time separately. Rising queue depth with stable processing latency usually indicates insufficient capacity. Rising processing time may indicate a downstream dependency, lock contention, allocation pressure, or pathological input. A flat queue can still be unhealthy if admission control rejects most work.

Memory retention is another risk. An array-based structure must clear removed references. A listener or retry queue may retain large request graphs. A task can also capture a heavyweight context in a lambda even when only a small identifier is needed. Heap analysis should follow retaining paths back to the queue and then ask why items are not leaving.

Recovery behavior needs the same precision. An in-memory queue loses pending work on process failure; a durable broker can redeliver it, usually with at-least-once rather than exactly-once semantics. Consumers must therefore decide where acknowledgement occurs and whether processing is idempotent. Acknowledging before the side effect risks loss; acknowledging after it risks duplicate execution if the process fails between the side effect and acknowledgement. Queue choice cannot be separated from the delivery contract.

Retries deserve special caution. Immediate requeue can create a hot loop and starve fresh work. Production retry queues normally apply delay, backoff, a maximum attempt count, idempotency, and a dead-letter path. Queue ordering and retry ordering must be designed together.

Common senior-level traps

- Choosing `Stack` by name without considering `ArrayDeque`.
- Treating a thread-safe queue as making a multi-step workflow atomic.
- Using an unbounded queue and calling it backpressure.
- Forgetting that `ArrayDeque` rejects `null`.
- Reading `PriorityQueue` iteration as sorted output.
- Storing values instead of indices in a monotonic deque, breaking expiry or duplicates.
- Using the wrong equality condition in a monotonic structure.
- Forgetting to clear removed references in a custom ring buffer.
- Claiming worst-case $O(1)$ for a two-stack queue when the guarantee is amortized.
- Designing insertion and removal but no overload or shutdown contract.

A strong interview answer

When asked to choose among these structures, begin with required order: newest-first, oldest-first, both ends, or best-priority-first. State capacity and overload behavior, then concurrency and blocking requirements. For an algorithmic problem, state the invariant before writing code. For a monotonic structure, explain why removed candidates can never become useful again and why each element enters and leaves at most once. For a production queue, discuss capacity, latency, rejection, observability, retries, and shutdown rather than stopping at Big-O notation.

Chapter summary

Stacks model nested unfinished work. Queues preserve a frontier and create a temporal handoff. Deques support both directions and enable efficient array-backed implementations. Circular buffers depend on explicit head, tail, and size invariants. Monotonic structures obtain linear performance by eagerly removing dominated candidates. In production, capacity, overload, fairness, concurrency, and shutdown are inseparable from ordering.

Revision Notes

- Use `ArrayDeque` for ordinary non-concurrent stacks and queues.
- Stack means LIFO; queue means FIFO; deque supports both ends.
- Prefer `offer/poll/peek` when empty or capacity conditions are normal outcomes.
- A circular queue needs unambiguous `head`, `tail`, and full/empty state.
- Clear removed references from custom array-backed generic structures.
- Bounded capacity requires a rejection, blocking, timeout, dropping, or spill policy.
- A thread-safe operation does not make a check-then-act sequence atomic.
- Define queue shutdown and whether accepted work is drained.
- A monotonic stack or deque stores only undominated candidates.
- Store indices when position, expiry, or duplicate identity matters.
- Each monotonic candidate is inserted and removed at most once: $O(n)$ total.
- Two-stack queues provide amortized $O(1)$, not worst-case $O(1)$ per operation.
- `PriorityQueue` is priority-ordered, not FIFO, and its iterator is not sorted.
- Monitor arrival rate, completion rate, wait time, oldest-item age, and rejection count.

Likely interview question: Why prefer `ArrayDeque` over `Stack`?

`ArrayDeque` provides a focused deque API, normally better locality and lower overhead, and avoids the legacy `Vector` inheritance of `Stack`. It is not synchronized; concurrent use requires a suitable concurrent structure or explicit synchronization.

Likely interview question: Why is a monotonic deque linear rather than quadratic?

Although a `while` loop may remove several elements during one iteration, every index is appended once and removed at most once. Across the entire input, total deque operations are therefore $O(n)$.

Likely interview question: How do you size a production work queue?

Start from acceptable queueing delay, arrival bursts, measured service rate, worker concurrency, and downstream capacity. Then choose a bounded capacity and explicit overload policy. Validate it with load tests and monitor wait-time percentiles and oldest-item age; capacity should not be chosen only to make rejection disappear.

Likely interview question: What does a blocking queue solve?

It provides thread-safe handoff and waiting semantics between producers and consumers. It does not by itself solve sizing, overload, retry, task idempotency, shutdown, or downstream bottlenecks; those remain system-design responsibilities.